

EXPERIMENT – 6

Experiment Title: Design and Implementation of a 4-Bit Common Bus System Using Multiplexers and Registers in Logisim Evolution

Aim: To design and implement a 4-bit common bus system using multiplexers and registers in Logisim Evolution, and to verify that data is correctly transferred between multiple registers through a shared bus governed by selection logic.

Software Used: Logisim Evolution

Theory:

Common Bus System

A common bus is a shared communication pathway that connects multiple components within a digital system, allowing data to be transferred between them in a controlled and efficient manner. Rather than dedicating individual data lines between every pair of components, a bus system uses a single shared line that all components can access, significantly reducing wiring complexity in larger systems.

In a register-based common bus system, multiple registers are connected to the same bus, but only one register is permitted to place its data onto the bus at any given time. This exclusive access is enforced using multiplexers, which act as the selection mechanism. A set of control bits, known as select lines, determines which register's output is forwarded onto the bus at any moment.

Role of the Multiplexer

A multiplexer is a combinational logic circuit that selects one of several input signals and forwards it to a single output line based on the value of its select inputs. In a 4-to-1 multiplexer, two select bits are used to choose among four input lines. In the context of a 4-bit common bus, one multiplexer is placed per bit position, and all four multiplexers share the same select lines. This ensures that when a particular register is selected, all four bits of that register appear simultaneously on the bus.

Data Transfer Mechanism

Once the desired source register's data appears on the bus, a destination register can latch it by asserting its load or write-enable signal on the rising edge of the clock. This two-step mechanism — source selection via the multiplexer and destination selection via the load signal — forms the core of register-to-register data transfer in a common bus architecture.

Why This Architecture Matters

The common bus design is foundational to processor design. In a CPU, the ALU, program counter, instruction register, and general-purpose registers all communicate through a shared internal bus. Understanding this architecture provides direct insight into how data flows within a processor during instruction execution.

Procedure:

The experiment was carried out using Logisim Evolution by constructing a 4-bit common bus system as described in the following steps.

Logisim Evolution was launched and a new project canvas was prepared for the circuit. A clock component was placed on the left side of the canvas, highlighted in the diagram, which serves as the common clock input driving all register operations throughout the circuit.

Four 4-bit registers were placed on the canvas and labelled R0, R1, R2, and R3 from top to bottom. Each register was pre-loaded with a distinct 4-bit binary value to ensure that transfers could be individually identified and verified. The values assigned were as follows: R0 was loaded with 0111, R1 with 1100, R2 with 0011, and R3 with 0101. These values were entered using constant input pins connected to the data input port of each register.

A separate enable or write-enable (WE) signal was connected to each register independently, allowing selective control over which register latches data from the bus at any given clock cycle. The clock signal from the common clock component was wired into the clock input of all four registers simultaneously.

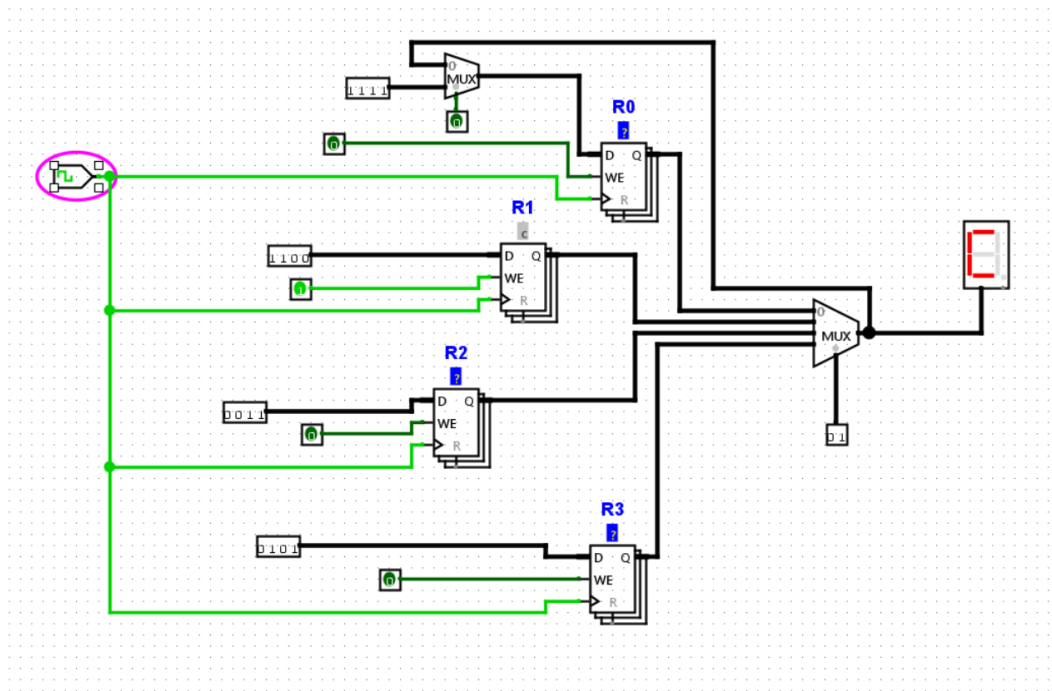
On the output side, the Q outputs of all four registers — R0, R1, R2, and R3 — were routed into a single 4-to-1 multiplexer placed to the right of the registers. This multiplexer acts as the bus selector, forwarding the output of exactly one register onto the common bus at a time based on the 2-bit select input provided at its bottom, which in the diagram is set to the value 11.

A separate single multiplexer was also placed at the top of the circuit connected to R0, which handles the individual bit-level selection for that register's input path, suggesting an additional layer of input routing for R0 specifically.

The output of the central 4-to-1 multiplexer was connected to a 7-segment display component placed on the far right of the canvas, which visually represents the value currently present on the common bus in real time.

The circuit was then tested by varying the 2-bit select input of the output multiplexer through all four combinations — 00, 01, 10, and 11 — and observing the corresponding register's data appearing on the 7-segment display output. The WE signal of a target register was then asserted on a clock edge to verify that bus data could be correctly latched into a destination register, completing the end-to-end transfer test across the common bus system.

Circuit Diagram:



Observations:

During simulation, the following behavior was consistently observed across all test cases. When the select lines were set to 00, the data stored in the first register appeared on the bus output. Changing the select to 01 immediately routed the second register's data onto the bus, and similarly, values 10 and 11 correctly forwarded the outputs of the third and fourth registers respectively. At no point did more than one register drive the bus simultaneously, confirming that bus contention was completely avoided through the multiplexer-based selection scheme.

When the load signal of a destination register was asserted during an active clock edge while a source register was selected, the bus data was latched correctly into the destination. Registers whose load signals were not asserted retained their original values unchanged, demonstrating proper isolation between registers sharing the same bus. The clock synchronization ensured that data was only captured at defined transition points, preventing any race conditions or unintended latching.

The observation table below summarizes the verified transfer behavior:

Select Lines (S1 S0)	Register on Bus	Bus Output (Example)
00	Register 1	Data of R1
01	Register 2	Data of R2
10	Register 3	Data of R3
11	Register 4	Data of R4

Results:

The simulation successfully demonstrated that data was routed correctly from each source register onto the common bus based on the select line input. Destination registers accurately latched the incoming bus data whenever their respective load signals were asserted on a rising clock edge. At any point in time, only one register drove the bus, which is consistent with the expected behavior of a properly functioning common bus architecture. All four possible select

combinations were tested and verified to produce the correct bus output without any errors or signal conflicts.

Conclusion:

The experiment successfully demonstrated the design and operation of a 4-bit common bus system using multiplexers and registers in Logisim Evolution. It was established that a multiplexer-based bus architecture provides a reliable and hardware-efficient method of enabling controlled data transfer between multiple registers without requiring dedicated point-to-point connections between each pair.

The use of a shared 2-bit select line across all four multiplexers ensured consistent and simultaneous selection of all four bits of the chosen register, while load signals provided precise control over which destination register captured the bus data. The clock signal further ensured that all transfers were synchronized and free from timing inconsistencies.

This experiment reinforced the understanding that common bus systems form a critical component of processor architecture, where multiple functional units must share a limited set of data pathways. The principles observed here — bus arbitration, multiplexed selection, and clocked data latching are directly applicable to the internal data path design of real-world CPUs and microcontrollers.

Author: Aayush Anand